DPDP COMPLIANCE AUDIT

odoo-hackathon-rewear

Digital Personal Data Protection Act 2023

54/100

Grade D — At Risk

Indicative penalty exposure: up to Rs 375 crore

DPDP Act 2023 Section 33 (max) — not legal advice

Scanned: 22 March 2026

Files indexed: 124

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

This codebase has 2 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 4 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 2.1–8.6 engineering days.

Total findings: 15
HIGH: 2
MEDIUM: 4
INFO: 4
PASS: 4

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	0/22 (0%)	FAIL
Section 6(6)	8	8/8 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	4.0/8 (50%)	WARN
Section 8(6)	8	8/8 (100%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	4/4 (100%)	PASS

Indicative Penalty Exposure

Up to ■375 crore

Maximum indicative penalties under DPDP Act 2023 Schedule for violations identified in this scan.

Section 33(3): up to ■375 crore

Indicative maximum penalties under DPDP Act 2023 Schedule / Section 33. Actual penalties are at regulatory discretion and depend on breach severity, intent, duration, and cooperation. Not legal advice.

Estimated Total Remediation Effort: 2.1–8.6 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: email, password

`index.js → badgeRoutes.js → roleMiddleware.js`

Flow 2: Application Logs

PII fields: email, firstname, pii_field

`userRoutes.js → roleMiddleware.js`

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
PASSWORD_NOT_HASHED	Section 8(1) — Security	HIGH	backend/models/index.js
AUDIT_TRAIL_PARTIAL	Section 8(4) — Audit Trail	MEDIUM	N/A
RETENTION_PARTIAL	Section 8(3) — Retention	MEDIUM	N/A
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	MEDIUM	N/A
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	MULTIPLE
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	backend/routes/index.js
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
DELETION_MECHANISM_PRESENT	Section 8 — Data Principal Rights	PASS	backend/routes/itemRoutes.js
BREACH_INDICATORS_ADEQUATE	Section 8(6) — Breach	PASS	N/A
DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	backend/routes/authRoutes.js
GRIEVANCE_OFFICER_PRESENT	Section 13 — Grievance Redressal	PASS	frontend/client/src/components/layout/Footer.tsx

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (5–19 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Password Not Hashed — models/index.js (1–3 hrs)

Week 2 — High Priority (10–42 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Audit Trail Partial (2–8 hrs)
- Retention Partial (2–8 hrs)
- Incomplete Deletion Coverage (2–8 hrs)
- Retention Missing — MULTIPLE (4–18 hrs)

Week 4 — Advisory (2–8 hours)

Address before next compliance review.

- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 17–69 hours (2.1–8.6 engineering days)

Details

Finding 1 of 15

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE:1

Evidence snippet: This file uses express.js to define API routes for badges.

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means that personal data may be processed without explicit user agreement, violating user privacy rights. This can lead to a loss of trust and potential legal penalties.

Fix:

1. Implement a consent management system to obtain and record user consent before processing personal data.
2. Integrate consent checks into API endpoints that handle personal data, ensuring data is only processed if consent is granted.
3. Add a mechanism to allow users to view, manage, and withdraw their consent.
4. Document the purposes for which consent is obtained and how data will be used.

DPDP:

Section 6 — Consent requires a clear and affirmative act by the data subject to signify agreement.

Example:

```
const express = require('express'); const router = express.Router(); // Middleware to check consent (example) const requireConsent = (req, res, next) => { if (req.user && req.user.hasConsented) { // Assuming user object has consent status next(); } else { res.status(403).json({ success: false, message: 'Consent is required' }); } }; router.post('/data', authenticateToken, requireConsent, (req, res) => { // Process data only if authenticated and consented res.json({ success: true, message: 'Data processed with consent' }); });
```

Finding 2 of 15

[Section 8(1) — Security]

2. [HIGH] PASSWORD_NOT_HASHED — backend / models / index.js:4

Evidence snippet: This file imports the User model, which is likely to contain user-related data.

Estimated fix time: 1–3 hours

■ Penalty exposure: up to ■250 crore (DPDP Section 33(3))

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Risk:

Storing passwords without proper hashing makes them vulnerable to breaches, violating the security obligations under DPDP. If the database is compromised, all user passwords could be exposed in plain text.

Fix:

1. Modify the User model definition to include a password hashing mechanism (e.g., using bcrypt) before saving.

2. Ensure all code paths that create or update user passwords implement this hashing logic.

3. If passwords are ever retrieved, ensure they are never exposed in plain text.

DPDP:

Section 8(1) — Security requires appropriate technical and organizational measures to protect personal data.

Example:

```
const bcrypt = require('bcrypt'); const SALT_ROUNDS = 10; // In your User model definition (e.g., User.js): User.beforeCreate(async (user) => { if (user.password) { user.password = await bcrypt.hash(user.password, SALT_ROUNDS); } }); User.beforeUpdate(async (user) => { if (user.changed('password')) { user.password = await bcrypt.hash(user.password, SALT_ROUNDS); } });
```

Finding 3 of 15

[Section 8(4) — Audit Trail]

3. [MEDIUM] AUDIT_TRAIL_PARTIAL — N / A:1

Evidence snippet: This file defines API routes using express.js.

Estimated fix time: 2–8 hours

■ Penalty exposure: up to ■25 crore (DPDP Section 33(3))

Partial audit trail detected. Found: change tracking. Missing: access logging, audit model.

Risk:

The lack of a comprehensive audit trail means that it's impossible to track who accessed or modified personal data, when, and what changes were made. This hinders accountability and makes it difficult to investigate security incidents or data breaches.

Fix:

1. Create a dedicated 'AuditLog' model to store details of data access and modifications.
2. Implement middleware that logs relevant information (user ID, action, timestamp, affected data) for all sensitive operations.
3. Ensure logs capture not only changes but also data access events.
4. Store audit logs securely and retain them according to policy.

DPDP:

Section 8(4) — Audit Trail requires maintaining records of processing operations involving personal data.

Example:

```
const AuditLog = require('../models/AuditLog'); // Assuming you create this model const logAudit = async (userId, action, details) => { await AuditLog.create({ userId, action, details, timestamp: new Date() }); }; // Example middleware to log access const auditMiddleware = (req, res, next) => { logAudit(req.user.id, 'ACCESS', { path: req.path, query: req.query }); next(); }; router.get('/users/:userId', authenticateToken, auditMiddleware, (req, res) => { // ... logic to get user data ... });
```

Finding 4 of 15

[Section 8(3) — Retention]

4. [MEDIUM] RETENTION_PARTIAL — N / A:4

Evidence snippet: This file imports the User model, which is likely to contain personal data.

Estimated fix time: 2–8 hours

Retention signals found in 4 model(s) but 5 PII model(s) have no retention policy. Uncovered: Item.js, Swap.js, SwapOffer.js.

Risk:

Failing to define retention policies for all models containing personal data means that data may be kept longer than necessary, increasing privacy risks and non-compliance. This violates the principle of data minimization and purpose limitation.

Fix:

1. For each identified model (Item, SwapOffer, Swap, and any others containing PII), define a clear data retention period.

2. Implement a mechanism (e.g., scheduled jobs, lifecycle policies) to automatically delete or anonymize data that has exceeded its retention period.

3. Document these retention policies and the rationale behind the chosen periods.

4. Update model definitions or associated services to enforce these retention rules.

DPDP:

Section 8(3) — Retention requires that personal data shall not be kept for longer than necessary for the purposes for which it is processed.

Example:

```
// Example for Item model (assuming Item.js exists and has a 'createdAt' field) const Item = sequelize.define('Item', { // ... other fields userId: { type: DataTypes.UUID, allowNull: false }, createdAt: { type: DataTypes.DATE, allowNull: false }, // Add a field to track retention, or use a separate service retentionEndDate: { type: DataTypes.DATE } }); // In a separate service or job scheduler: async function enforceRetention() { const cutoffDate = new Date(); cutoffDate.setMonth(cutoffDate.getMonth() - 12); // Example: 12-month retention await Item.destroy({ where: { retentionEndDate: { [Op.lt]: cutoffDate } } }); }
```

Finding 5 of 15

[Section 8 — Data Principal Rights]

5. [MEDIUM] INCOMPLETE_DELETION_COVERAGE — N / A

Estimated fix time: 2–8 hours

■ Penalty exposure: up to ■50 crore (DPDP Section 33(3))

Deletion mechanism detected but may not cover all PII storage. Uncovered files: backend/config/database.js, backend/models/Badge.js, backend/models/EcoImpact.js, backend/models/Item.js, backend/models/Redemption.js. DPDP Section 8 requires complete erasure of all personal data across all systems when a user requests deletion.

Risk:

Failure to implement a complete data deletion mechanism means personal data may persist longer than necessary, violating data minimization principles and increasing the risk of unauthorized access or misuse.

Fix:

1. Implement a dedicated API endpoint (e.g., DELETE /users/:userId) that triggers a cascade of deletion operations across all relevant data stores.

2. In the user model (e.g., backend/models/User.js), define a 'beforeDestroy' hook to ensure associated data in other models (like Badges, Items, EcoImpacts) is also marked for deletion or deleted.

3. Create a service or controller function to orchestrate the deletion process, ensuring all PII is targeted for removal from databases, logs, and any other storage.

4. Add comprehensive unit and integration tests to verify that user data is completely removed upon request, covering all identified PII storage locations.

DPDP:

Section 8 — Data Principal Rights

Example:

```
router.delete('/:userId', authenticateToken, RoleMiddleware.isAdmin, userIdValidation, validateRequest, userController.deleteUser); // In userController.js exports.deleteUser = async (req, res) => { const { userId } = req.params; try { // Logic to find user and trigger cascade delete const deleted = await User.destroy({ where: { id: userId }, cascade: true }); if (deleted) { res.status(200).json({ success: true, message: 'User data deleted successfully' }); } else { res.status(404).json({ success: false, message: 'User not found' }); } } catch (error) { res.status(500).json({ success: false, message: 'Error deleting user data', error: error.message }); } };
```

Finding 6 of 15

[Section 8(3) — Retention]

6. [MEDIUM] RETENTION_MISSING — MULTIPLE:1

Evidence snippet: This file defines the Badge model using Sequelize, which is a data model.

Estimated fix time: 4–18 hours

■ Penalty exposure: up to ■50 crore (DPDP Section 33(3))

Affected files (5):

- backend/models/Item.js
- backend/models/Swap.js
- backend/models/SwapOffer.js
- backend/models/User.js
- backend/models/index.js

Model/schema file contains PII fields but no retention or expiry signals detected. Detected in 5 files: Item.js, Swap.js, SwapOffer.js, User.js and 1 more

Risk:

Absence of retention policies means personal data may be stored indefinitely, increasing the risk of unauthorized access and violating data minimization principles.

Fix:

1. Add a 'retentionPeriodDays' field to the Badge model schema (e.g., backend/models/Badge.js) to specify how long badge data should be kept.

2. Implement a scheduled job (e.g., using node-cron) that runs daily to identify and delete badges older than their defined retention period.

3. Modify the deletion logic in related models (e.g., User model) to also consider the retention periods of associated data like badges.

4. Document the retention periods and the rationale for each in a data management policy.

DPDP:

Section 8(3) — Retention

Example:

```
const Badge = sequelize.define('Badge', { // ... other fields awardedAt: { type: DataTypes.DATE, defaultValue: DataTypes.NOW }, retentionPeriodDays: { type: DataTypes.INTEGER, allowNull: true, // Or set a default retention period defaultValue: 365 // Example: 1 year retention } }, { tableName: 'badges', hooks: { ... } }); // In a scheduled job (e.g., using node-cron): const cutoffDate = new Date(); cutoffDate.setDate(cutoffDate.getDate() - retentionPeriodDays); // Needs to be dynamic per badge or global await Badge.destroy({ where: { awardedAt: { [Op.lt]: cutoffDate } } });
```

Finding 7 of 15

[Section 11 — Data Portability]

7. [LOW] DATA_PORTABILITY_MISSING — N / A

Estimated fix time: 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The absence of a data portability feature prevents users from accessing their personal data in a usable format, hindering their ability to exercise their rights under DPDP.

Fix:

1. Create a new API endpoint (e.g., GET /users/:userId/data) that is protected by authentication.
2. In the corresponding controller, fetch all personal data associated with the user from various data models (e.g., Badge, Item, EcoImpact).
3. Format the collected data into a machine-readable format, such as JSON or CSV.
4. Return the formatted data in the API response, allowing the user to download or view it.

DPDP:

Section 11 — Data Portability

Example:

```
router.get('/:userId/data', authenticateToken, userController.exportUserData); // In
userController.js exports.exportUserData = async (req, res) => { const { userId } =
req.params; try { const user = await User.findByPk(userId, { include: [...] }); // Include all
relevant models // Aggregate data from user and associated models const userData = { /* ...
structure user data ... */ }; res.setHeader('Content-Type', 'application/json');
res.setHeader('Content-Disposition', `attachment; filename=user_data_${userId}.json`);
res.status(200).send(JSON.stringify(userData, null, 2)); } catch (error) {
res.status(500).json({ success: false, message: 'Error exporting user data', error:
error.message }); } };
```

Finding 8 of 15

[Section 7 — Legitimate Use]

8. [INFO] INTERNAL_ROUTE_PII_ACCESS — backend / routes / index.js

Internal file (index.js) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 9 of 15

[Section 5 — Notice]

9. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 10 of 15

[Section 5 — Purpose Limitation]

10. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 11 of 15

[Section 16 — Cross-Border Transfer]

11. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 12 of 15

[Section 8 — Data Principal Rights]

12. [PASS] DELETION_MECHANISM_PRESENT — backend / routes / itemRoutes.js

Data deletion mechanism detected. Found 5 deletion endpoint(s) and 25 deletion signal(s).
Note: Verify deletion cascades to all PII tables (sessions, logs, related records).

Fix:

1. Verify deletion cascades to: sessions, audit_logs, third-party sync records.
2. Use database CASCADE DELETE or explicit multi-table deletion in deletion service.
3. Test deletion completeness: create account, delete, query all PII tables for orphaned data.

Finding 13 of 15

[Section 8(6) — Breach]

13. [PASS] BREACH_INDICATORS_ADEQUATE — N / A

Logging and/or breach alerting present.

Finding 14 of 15

[Section 11 — Right to Access]

14. [PASS] DATA_ACCESS_PRESENT — backend / routes / authRoutes.js

Data access endpoint detected for authenticated users.

Finding 15 of 15

[Section 13 — Grievance Redressal]

15. [PASS] GRIEVANCE_OFFICER_PRESENT — frontend / client / src / components / layout / Footer.tsx

Grievance Officer or Data Protection Officer contact information detected in codebase.

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Missing age verification for user accounts Confidence: 45%

Section 9 — Children

The rule engine did not report any checks for age verification. The repository context indicates `auth_mechanism: "not detected"`, suggesting a lack of robust user account management where age verification would typically occur before processing children's data.

Recommendation: Implement an age verification mechanism during any user interaction that could lead to processing children's data, especially if user accounts are created or PII is collected. Add specific code to check age before data collection.

[AI-INFERRED] Unchecked third-party data sharing Confidence: 50%

Section 5 — Purpose Limitation

The rule engine explicitly states `third_party_checked: false`, indicating no automated checks were performed for data sharing with third parties. The repository context is empty, so no specific third-party services were identified for manual review.

Recommendation: Implement automated checks to identify data transfers to third-party services (e.g., by scanning for API calls to external domains, SDKs, or data export functions). Ensure these transfers align with stated purposes and consent.

[AI-INFERRED] Missing DPO contact information Confidence: 35%

Section 10 — Data Protection Officer

The rule engine coverage does not indicate any checks for the presence or accessibility of Data Protection Officer (DPO) contact information. The repository context is too limited to confirm its absence in code.

Recommendation: Add a clearly visible and accessible contact point for the Data Protection Officer (or equivalent grievance officer) within the application's public-facing interfaces (e.g., privacy policy, contact page, or 'About Us' section).

[AI-INFERRED] Missing user data access view Confidence: 45%

Section 8 — Right to Access

While the rule engine identified `DATA_PORTABILITY_MISSING` (downloading data), it did not explicitly check for a user-facing interface or mechanism allowing data principals to *view* all their personal data processed by the application. The `auth_mechanism: "not detected"` in the repository context suggests a lack of user accounts or a dashboard where such a feature would reside.

Recommendation: Develop a secure, authenticated user interface or API endpoint where data principals can view all personal data associated with them, ensuring transparency and enabling them to verify accuracy.

Deep Compliance Review

Overall Risk: CRITICAL

The current architecture presents critical DPDP compliance risks due to pervasive PII exposure across multiple layers, including development logs, error messages, and user-generated content. Significant gaps exist in data minimization principles, leading to uncontrolled PII storage and premature disclosure, alongside inadequate audit logging for critical authentication events. These issues collectively undermine the organization's ability to ensure data security, maintain user consent, and demonstrate accountability under DPDP regulations. Immediate action is required to prevent potential data breaches and regulatory penalties.

E

Most Critical Action: Implement a centralized logging solution that automatically redacts or masks PII from all log outputs, especially in development and non-production environments.

Layer Breakdown

Configuration & Secrets — 2 finding(s). The Configuration & Secrets layer exhibits two DPDP compliance risks primarily within the Swagger API definition. One high-severity issue involves the potential exposure of PII through verbose error messages, directly impacting data security. A medium-severity issue relates to the undefined schema for user preferences, which could lead to uncontrolled storage of personal data, undermining data minimization and purpose limitation principles. Other configuration files do not present direct DPDP compliance concerns.

API Routes & Controllers — 2 finding(s). The API Routes & Controllers layer for authentication handles PII collection and critical security operations. While strong password validation is in place, there are architectural gaps related to data minimization for mandatory PII collection and a lack of explicit audit logging for key authentication events, which directly impacts DPDP compliance for consent, purpose limitation, security, and accountability.

Data Models & Storage — 4 finding(s). This layer exhibits critical DPDP compliance risks related to PII exposure in development logs and the lack of controls over user-generated content, which could inadvertently store PII. Additionally, architectural choices regarding user ID types and silent error handling present medium-level risks to data security and accountability. Addressing these issues is crucial for strengthening data protection posture.

Authentication & Session — 4 finding(s). The Authentication & Session layer exhibits critical compliance risks related to insecure error handling, which undermines auditability and system stability. A significant architectural gap exists in token invalidation on logout, posing a direct threat to PII security. Additionally, data minimization principles are violated by attaching excessive PII to request objects, increasing the risk of unintended data exposure.

Third-Party & PII Flows — 4 finding(s). This layer exhibits several DPDP compliance risks primarily related to PII over-sharing and inadequate controls over data exposure. Direct PII disclosure in API responses, potential information leakage via error messages, and the lack of safeguards for user-submitted PII in free-text fields are significant concerns. Additionally, the public exposure of the application's PII data model through API documentation presents an architectural vulnerability.